How to setup everything to successfuly run CWatM

Danube Water Balance

Martin Bednář

Last updated: October 30, 2024

Contents

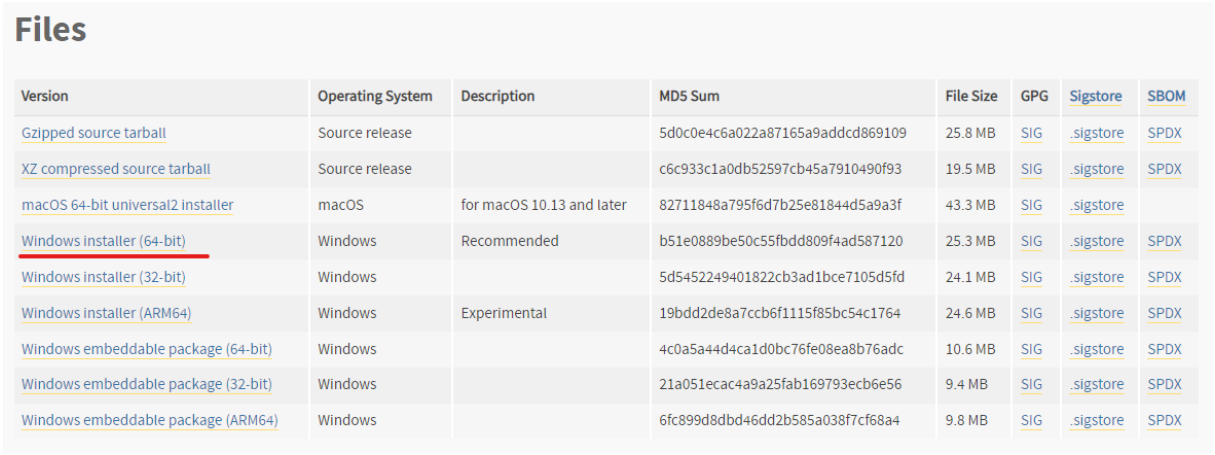
1 Python setup

The following text describes instructions regarding the Windows 10 operating system (OS). However, there might be some differences in case of different OS.

The first step is to install the Python language in your machine. You can find the latest version on the official Python Website. As of October 2024, the latest version of Python is 3.13.0. However, it is a relatively new release, thus not all packages might be up to date. So scroll a bit down and download the second latest version which should be **3.12.7**.

1.1 Python installation

After selecting the particular Python version you should be redirected to a new webpage with some information about this release features. Scroll down till you find a table (Figure 1) with version files. Find a 64-bit version for Windows OS and click its name. The download should begin.



Files

Version	Operating System	Description	MD5 Sum	File Size	GPG	Sigstore	SBOM
Gzipped source tarball	Source release		5d0c0e4c6a022a87165a9addcd869109	25.8 MB	SIG	.sigstore	SPDX
XZ compressed source tarball	Source release		c6c933c1a0db52597cb45a7910490f93	19.5 MB	SIG	.sigstore	SPDX
macOS 64-bit universal2 installer	macOS	for macOS 10.13 and later	82711848a795f6d7b25e81844d5a9a3f	43.3 MB	SIG	.sigstore	
Windows installer (64-bit)	Windows	Recommended	b51e0889be50c55fbdd809f4ad587120	25.3 MB	SIG	.sigstore	SPDX
Windows installer (32-bit)	Windows		5d5452249401822cb3ad1bce7105d5fd	24.1 MB	SIG	.sigstore	SPDX
Windows installer (ARM64)	Windows	Experimental	19bdd2de8a7ccb6f1115f85bc54c1764	24.6 MB	SIG	.sigstore	SPDX
Windows embeddable package (64-bit)	Windows		4c0a5a44d4ca1d0bc76fe08ea8b76adc	10.6 MB	SIG	.sigstore	SPDX
Windows embeddable package (32-bit)	Windows		21a051ecac4a9a25fab169793ecb6e56	9.4 MB	SIG	.sigstore	SPDX
Windows embeddable package (ARM64)	Windows		6fc899d8dbd46dd2b585a038f7cf68a4	9.8 MB	SIG	.sigstore	SPDX

Figure 1: *Python installation file location and download.*

Open the file and follow recommended instructions. **Be careful during installation of the option to add Python into PATH environmental variable**. Figure 2 shows what to look for during the installation. It is not the end of the world if you did not check this checkbox and it is possible to add Python into PATH variable later. However, try to do it now as it will make your life way simpler during using python. After successful installation open the Command Prompt (use i.e. WIN + R shortcut to open Run window and write *cmd* and hit ENTER.). If you write *python -V* and hit ENTER. It should return the Python version number you just installed (Figure 3).

Now check if the Python work using the famous phrase "Hello world!". Write in the

Figure 2: *Do not forget to check the "Add python.exe to PATH" option.*

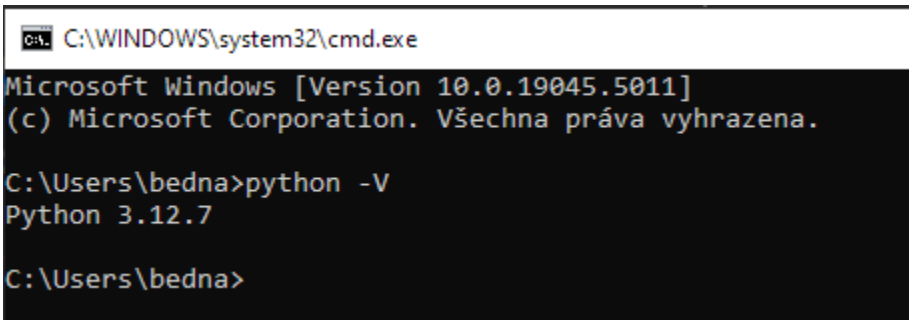


Figure 3: *Checking Python version in the Command Prompt environment*

Command Prompt *python* and hit ENTER. You should enter the python environment (Figure 4).

Now write *print("Hello world!")* and hit ENTER. The print function in the python just prints the string you enter as an argument inside it. Hence you should see the printed text "Hello world!". It should work if everything was installed properly. Otherwise you will need to troubleshoot the Python installation.

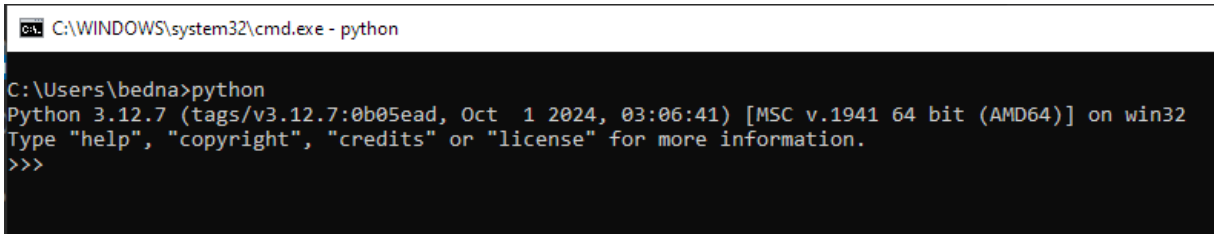


Figure 4: *Entering the Python environment in the Command Prompt.*

1.2 Virtual environment setup

1.2.1 What is virtual environment

The virtual environment enables you to run a clean "separate copy" of Python. Then you are able to install packages into this copied python which do not collide which then "main" Python installed.

For example, you install Python on your machine and then install a package named *thebestpackage*. Lets say it is a new package, thus its version would be 1.0.0. Then you write a code using this package's functions. After some time you will need to update this package because for example in the version 1.2.0 the creator added a new feature you would like to use. So you update the package. However, the new version changed how some existing function operated and now your former code does not work properly. However, if you installed the first version of package in the virtual environment the package would remain at version 1.0.0 even if you update the package on the main Python because the virtual environment created a separate copy od Python.

Therefore, the virtual environment is very usefull in controling the versions of packages used for your particular peace of code and if you will need and updated version for different code (on main Python version or in different virtual environments) it will remain functional. **Nevertheless, the virtual environment is not necessary and is totally optional so if you are not interested you can simply skip this section and install all Python packages into your main Python environment.**

1.2.2 How to install virtual environment

There are many ways to setup an virtual environment. Two ways of setup the virtual environment will be described (The second one was presented in the CwatM Level A1 Summer School). **The first one** is using the *venv* package that is built-in the Python since version 3.3. The usage of this package is following:

1. Create a folder for your project.

2. Open the Command Prompt inside this folder.
3. Use command: *python -m venv .venv*

Now you have create a virtual environment that is named ".venv" inside your folder. So, how does that command works? Firstly you tell it to use Python (*python*) then the flah *-m* tells python to execute a module as script and the module would be *venv* which follows. The last part is simply the name of your virtual environment. The name of virtual environment can be whatever you like, e. g. "my_project" or you can use a path to any folder you like if you do not want to create a virtual environment in the destination you are currently in.

To activate the virtual environment using the *venv* package you need to open a Command Prompt inside your project folder (where the virtual environment was created) and use following command (".venv" is the name of your virtual environment):

.venv/Scripts/Activate.bat

This will start the virtual environment and you will see its name in brackets before the next command line (Figure 5). After you activate the virtual environment all the

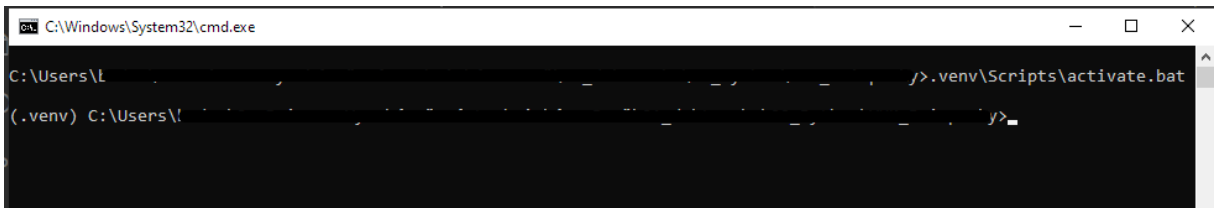


Figure 5: *Activating the virtual environment named ".venv"*

packages you install will be installed specificaly in this activated environment and will not be affected by any changes in the main Python environment. To deactivate the virtual environment simply put in Commant Prompt the *deactivate* command.

The second option (that is described here) is to use another package that works the same way but as a wrapper around the *venv* package. It's named *virtualenvwrapper-win* and you need to install it first. To install this package (or any package) you can use *pip*. The following command will install this package for you. Open the Command Prompt and write:

pip install virtualenvwrapper-win

It should start the installation of the package and if no error occurs it should say "Successfuly installed . . . " at the bottom of the Command Prompt. To create a virtual environment using this package you simply write following commnd and in the Command Prompt:

mkvirtualenv .venv

Similarly to the previous option the ".venv" will be the name of your virtual environment and it can be whatever you like. It should create the virtual environment and activate is as well (again you will see the name of the environment in brackets at the start of the command line). Activating this virtual environment can be later done using the command

workon .venv

where ".venv" is the name of your environment.

1.3 Python package installation

You will use aforementioned *pip* to install any package you like. You should be inside the created and activate virtual environment so the packages will be installed separately from main Python. The packages you will need to run the CWatM are following:

- numpy
- scipy
- netcdf4
- pandas
- xslxwriter

To install them you use the same command as was used to install the virtual environment package:

pip install package_name

There is a one package named *GDAL* that needs to be installed manually. Firstly, you need to download a wheel file (wheel is a type of file which starts the installation of the package) from this Github repository. Now scroll down and download a **.whl* file of *GDAL* for your specific Python and Windows bit version. For 64bit OS and Python 3.12.7 the file's name shoud be "**GDAL_-3.9.2-cp312-cp312-win_amd64.whl**". The manual installation is made using the same *pip install* command excel instead of the package name you enter the path to the downloaded WHL file including the file's name and extension.

pip install "path_to_file"\GDAL_-3.9.2-cp312-cp312-win_amd64.whl

In case that the Command Prompt is opened in the same directory as the WHL file you can skips the "path_to_file" section and simply write just the file's name with extension.

2 CWatM setup

CwatM is provided in the working condition in the first calibration run for all pilot basins. The files can be accessed via the project's sharepoint web where all project partners should have access to.

2.1 Download CWatM files

There are two parts of files you need to downlaod from said sharepoint. First is the CWatM itself and second is a folder with all the data needed to run to model. Under the sharepoint directory ***Documents\General\SO2\working*** there is a folder named "CWatM_model_developversion". This folder contains the working python files representing the CWatM which will actually do the computation. Thus, download this folder into your project folder.

Afterwards, in directory ***Documents\General\SO2\working\CWatM_1min*** there are files needed to run the model for all the pilot basins. For example, the basin Morava is what you are interested in. Thus, download the whole folder named *Morava* into your project folder. You might need to download the files in small batches as the Sharepoint tends to download large folders in zipped batches which might lead to an incomplete download of the large raster maps.

2.2 Update the settings file

After downloading all the files open the file *setting_basinname_1min.ini* which is in "settings" folder inside the basin folder. Scroll down till you find the section titled as **[FILE_PATHS]**. There you need to check if the path variables corresponds to location of the basin files.

2.3 Running the simulation

After all is set up the running of the model is very easy. Basically you run the *runcwatm1.bat* file inside the *setting* directory for your basin. In case you want to run the virtual environment you need to edit this batch file. To do so, open the *runcwatm1.bat* in text editor and add a line on the first row. This line should be the command for activating the virtual environment. In case of the first package for virtual environment the command is:

call .venv\Scripts\Activate.bat

for the second method it's:

workon .venv

The ".venv" is the name of th virtual environment you created for the project.

And that's it, folks! Enjoy your modelling adventure.